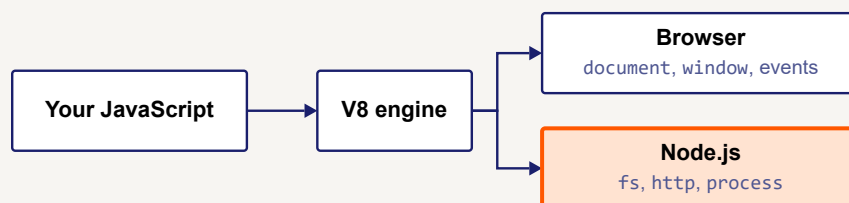


1. **The Node.js runtime:** What Node.js is: JavaScript outside the browser · Running code: node file.js, the REPL and --watch · A first server: the http module, ports and listen
2. **Modules:** Modules: require and module.exports · What require finds: built-in modules, packages and your files · ES modules: import, export and "type": "module" · Files and paths: fs, path and __dirname
3. **npm and package.json:** package.json: the project's manifest · Installing packages: dependencies, devDependencies and node_modules · Version numbers and ranges: what ^4.19.0 means · package-lock.json, npm ci and what to commit
4. **Asynchronous code:** Blocking and non-blocking: why Node code is asynchronous · Callbacks and the error-first convention · Promises: a value that arrives later · async and await · Errors in async code: try/catch every await · Independent work in parallel: Promise.all
5. **The event loop:** The event loop: one thread, a call stack and queues · Microtasks before tasks: predicting the order · Never block the event loop
6. **Configuration and app structure:** Environment variables and process.env · .env files, config and keeping secrets out of Git · Why a framework, and why Express · A Node/Express project layout

The Node.js runtime

What Node.js is: JavaScript outside the browser

Node.js is a runtime: a program that runs JavaScript outside the browser, on your computer or on a server.



One engine; the APIs around it make the difference.

What your code can reach

	In the browser	In Node.js
The page	document, window, DOM events, alert, localStorage	None: document is not defined
Files and network	No access to your disk; only fetch and uploads	fs (files), http (servers), path, os
The running program	location, navigator	process: process.env, process.argv, process.exit()
Global object	window (also globalThis)	globalThis

Why JavaScript on a server

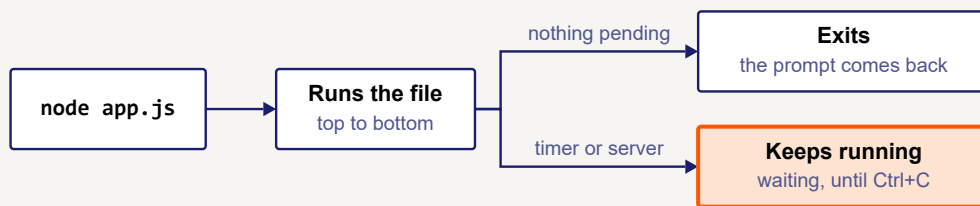
- **One language** for the page and the API, and data travels as JSON, which is JavaScript's own object notation.
- **Built for waiting:** a web server spends most of its time waiting for the database, the disk or another API, and Node keeps working while it waits (see [Blocking and non-blocking](#)).

Example. Save hello.js with `console.log(process.version);` and run `node hello.js`: it prints the version, such as v24.11.0. Add `document.title = 'x';` and it stops with `ReferenceError: document is not defined`. These pages target **Node 24 LTS** (Long-Term Support: about three years of fixes, the safe choice to deploy).

Common mistake. Calling Node a language or a framework. JavaScript is the language, Node is the **runtime**, and Express is a **framework** that runs on Node. The same confusion leads to pasting browser code (`document.querySelector`, `alert`) into a server file: a server has no page and no user in front of it.

Running code: node file.js, the REPL and --watch

`node file.js` runs a file from a terminal: top to bottom, then it stays alive only while something is left to wait for.



A program ends when nothing is left to wait for.

node file.js	Runs a file (<code>node file</code> also works: <code>.js</code> is assumed).
node	Opens the REPL (Read–Eval–Print Loop): type an expression and see its value, as in the browser console. Leave with <code>.exit</code> .
node --watch file.js	Restarts the program every time you save. Without it, a running program ignores your edits: it keeps the code it loaded at start-up. Built in since Node 22 (it replaces the nodemon package).
Ctrl+C	Stops the program running in that terminal (press it twice to leave the REPL).
process.argv	The words typed on the command line, as strings. <code>process.argv[2]</code> is the first word after the file name.

```

// greet.js
const name = process.argv[2] || 'stranger';
console.log(`Hello, ${name}!`);

// In the terminal:
// $ node greet.js Ana
// Hello, Ana!
// $ node greet.js
// Hello, stranger!
// $ node
// > 2 ** 10
// 1024
// > .exit
  
```

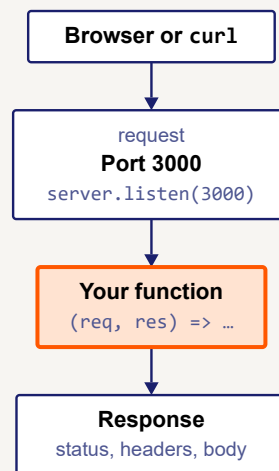
Example. Run `node --watch greet.js Ana`, change `Hello` to `Hi` and save: the terminal reports a restart and prints `Hi, Ana!`, without retyping the command. Projects keep that command in an npm script, so `npm start` runs it (see [package.json](#)).

Common mistake. Editing a server's code, retrying and seeing no change. The server was started with plain `node`, so it still runs the code it loaded at start-up. Stop it with `Ctrl+C` and start it again, or run it with `--watch`.

A first server: the http module, ports and listen

A **server** is a program that waits on a **port** for requests and answers each one; Node's built-in `http` module is

enough to write one.



Every request reaches one function, which builds the response.

```
// server.js
// Node's built-in http module
const http = require('node:http');

const server = http.createServer((req, res) => {
  if (req.method === 'GET' && req.url === '/health') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ status: 'ok' }));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Not found' }));
  }
});

server.listen(3000, () => console.log('Listening on http://localhost:3000'));
```

- req** The request: `req.method` (GET, POST...) and `req.url` (the path and query, such as `/health`).
- res** The response you build: `res.writeHead(status, headers)`, then `res.end(body)` sends it. A request whose response never ends leaves the client waiting.
- `require('node:http')`** Loads a module that comes with Node; the `node:` prefix marks it as built in (see [Modules](#)).

Example. Run `node server.js` and open `http://localhost:3000/health`: the browser shows `{"status":"ok"}`. Any other path answers 404. From a terminal, `curl http://localhost:3000/health` (a tool that sends a request and prints the response) shows the same. Each new route is one more `if`, which is why frameworks exist (see [Why a framework](#)).

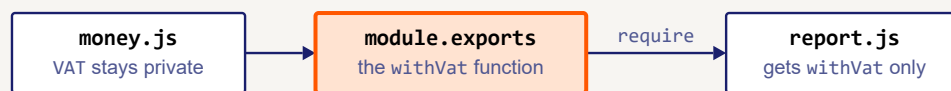
Common mistake. Starting the server again in a second terminal while the first is still running. The second one fails with Error: `listen EADDRINUSE: address already in use :::3000`: only one program can listen on a port. Stop the first with Ctrl+C in its terminal, or use another port.

Modules

Modules: `require` and `module.exports`

In Node every file is a **module** with its own private scope: it shares a value by assigning it to `module.exports`,

and another file gets that value with `require('./path')`.



Only what a file puts on `module.exports` leaves it.

Two shapes of export

	In the exporting file	In the importing file
One thing	<code>module.exports = withVat;</code>	<code>const withVat = require('./money');</code> (you choose the name)
Several things	<code>module.exports = { listTasks, createTask };</code>	<code>const { listTasks } = require('./tasks');</code> (see What require finds)

- **Paths** to your own files start with `./` (this folder) or `../` (the parent folder); `.js` can be left out.
- **Runs once**: a module's code runs the first time it is required. Node keeps (caches) its `module.exports` and gives the same value to every later `require`.
- **Last assignment wins**: `module.exports = value` replaces whatever the file exported before.
- `exports` **is only a shortcut** to the same object: `exports.total = 1` works, `exports = { total: 1 }` does not, because it only re-points the shortcut (the Try it box shows why).

```

// src/utils/money.js
// private: not exported
const VAT = 0.21;
function withVat(price) {
  return Math.round(price * (1 + VAT) * 100) / 100;
}
// export ONE function
module.exports = withVat;

// src/report.js
const withVat = require('./utils/money'); // ./ = relative to THIS file
console.log(withVat(100));
// → 121
console.log(typeof VAT);
// → 'undefined': VAT stayed private
  
```

```

// How Node runs each file (simplified): your code is wrapped in a
// function that receives "module" and "exports".
function runModule(code) {
  const module = { exports: {} };
  code(module, module.exports);
  // what require() gives back
  return module.exports;
}

const good = runModule((module) => {
  module.exports = { greet: (name) => 'Hi ' + name };
});
console.log(good.greet('Ana'));
// → Hi Ana

const bad = runModule((module, exports) => {
  exports = { greet: (name) => 'Hi ' + name }; // re-points the shortcut only
});
console.log(bad);
// → {}: nothing was exported
  
```

Example. `store.js` holds `const tasks = []`; `module.exports = tasks`; `add.js` requires it and pushes a task; `list.js` requires `./store` and sees that task. The module ran once, so there is **one** array shared by every file, not one copy per require.

Common mistake. Forgetting the export. If `money.js` defines `withVat` but never assigns `module.exports`, `require('./utils/money')` returns an empty object `{}`, and `withVat(100)` throws `TypeError: withVat is not a function`. When a required value is `{}` or `undefined`, look at the last lines of the file you required.

What require finds: built-in modules, packages and your files

The string you pass to `require` is an address: its first characters tell Node whether to load a **built-in module**, a **package** from `node_modules/` or **one of your own files**.

Three kinds of `require`

You write	Node loads	Install first?
<code>require('node:fs')</code>	The built-in file-system module	No: part of Node
<code>require('express')</code>	A package : <code>node_modules/express/</code> , searching up the folders	Yes: <code>npm install express</code>
<code>require('./app')</code>	<code>app.js</code> (or <code>app/index.js</code>) in the folder of <i>this</i> file	No: it is your file
<code>require('../lib/tasks')</code>	Up one folder, then <code>lib/tasks.js</code>	No

- The **node:** **prefix** is optional (`require('fs')` works too) but recommended: it shows at a glance that the module comes with Node and is not an npm package.
- A **bare name is always a package**. Node looks in `node_modules/` next to the file, then in each parent folder. If the package was never installed, the first line fails with `Error: Cannot find module 'express'`.
- A **path** starts with `./`, `../` or `/`, and is relative to the file that contains the `require`, not to the folder you started Node in.

```
// What a tasks module exports: an object with two functions
const tasks = {
  listTasks() { return 'listing tasks'; },
  createTask() { return 'creating a task'; },
};

// Destructuring takes properties out by name
const { listTasks, createTask } = tasks;
// listing tasks
console.log(listTasks());
// creating a task
console.log(createTask());

// A name that is not a property gives undefined
const { deleteTask } = tasks;
console.log(deleteTask);
// → undefined
```

Example. You add `deleteTask` to `const { listTasks, deleteTask } = require('./tasks')`; but forget it in the `module.exports = { ... }` of `tasks.js`. The variable is `undefined`, and the first call fails with `TypeError: deleteTask is not a function`. Check the export list of the file you required.

Common mistake. Writing `require('app')` for your own `app.js`. Without `./`, Node reads `app` as a **package** name, searches `node_modules/`, and fails with `Cannot find module 'app'`, even though `app.js` sits next to the file. Paths to your own files always start with `./` or `../`.

ES modules: import, export and "type": "module"

ES modules (ESM) are the module syntax built into the language, `import` and `export`; **browsers and front-end tools such as Vite use them, and Node supports them next to CommonJS**.

The same module in both systems

	CommonJS (server examples here)	ES modules (browsers, Vite, React)
Export one thing	<code>module.exports = app;</code>	<code>export default app;</code>
Export several	<code>module.exports = { a, b };</code>	<code>export function a() {}</code> or <code>export { a, b };</code>
Import one thing	<code>const app = require('./app');</code>	<code>import app from './app.js';</code>

	CommonJS (server examples here)	ES modules (browsers, Vite, React)
Import some	<code>const { a } = require('./lib');</code>	<code>import { a } from './lib.js';</code>

Which system a file uses

- .cjs** Always CommonJS.
- .mjs** Always ES module.
- .js** Follows the "type" field of the nearest package.json: "commonjs" (what npm init -y writes) or "module". With no "type", Node guesses: a file that contains import or export is re-run as an ES module, with a warning.

What changes in an ES module

- Full file names** in relative imports: `import app from './app.js'`, not `./app'`.
- No CommonJS names:** `require`, `module.exports`, `__dirname` and `__filename` do not exist; `import.meta.dirname` gives the folder.
- Top-level await** is allowed: `await` outside any function.

Example. You paste `import express from 'express';` from a tutorial into an `app.js` that uses `require` everywhere else. With "type": "commonjs", Node stops with `SyntaxError: Cannot use import statement outside a module.` With no "type", it re-runs the file as ESM and fails on the next line: `ReferenceError: require is not defined in ES module scope.` Fix: write the line in the project's style, `const express = require('express');`.

Common mistake. Adding "type": "module" to package.json to make one copied import work. Every .js file of the project is then read as ESM, so every `require` and `module.exports` breaks at once. Pick one system per project and never mix the two in one file.

Files and paths: fs, path and __dirname

The built-in `node:fs` module reads and writes files, `node:path` builds file paths, and `__dirname` is the folder of the current file.

The same line in `src/load.js`, started from two folders

Node started in	'tasks.json' opens	<code>path.join(__dirname, 'tasks.json')</code> opens
<code>src/</code>	<code>src/tasks.json</code>	<code>src/tasks.json</code>
The project folder	<code>tasks.json</code> in the project folder: <code>ENOENT</code> if there is none	<code>src/tasks.json</code>

- fs.readFileSync(file, 'utf8')** Reads a file as text and waits until it is done. Fine at start-up; never inside a request handler (see [Never block the event loop](#)).
- fs.promises.readFile(file, 'utf8')** The same without waiting: it returns a promise, used with `await` (see [async and await](#)).
- fs.writeFileSync(file, text)** Creates the file, or replaces its contents.
- path.join('data', 'tasks.json')** Joins parts with the separator of the operating system: `/` on macOS and Linux, `\` on Windows.
- JSON.parse(text)** Turns a JSON file's text into a value; `JSON.stringify(value, null, 2)` writes it back (see [JSON](#)).
- __dirname** The folder of the current file, in CommonJS. In an ES module: `import.meta.dirname`.

```
// src/load.js
const fs = require('node:fs');
const path = require('node:path');

// next to this file, wherever Node starts
const file = path.join(__dirname, 'tasks.json');
const tasks = JSON.parse(fs.readFileSync(file, 'utf8'));
tasks.push({ id: tasks.length + 1, title: 'Write the report', done: false });
fs.writeFileSync(file, JSON.stringify(tasks, null, 2));
```

```
// → null, 2: indent by two spaces
console.log(tasks.length, 'tasks saved');
```

Example. Running `node src/load.js` from the project folder and `node load.js` from inside `src/` both find `src/tasks.json`, because the path is built from `__dirname`. With the plain `'tasks.json'`, the first command fails with Error: ENOENT: no such file or directory, and the full path in the message shows where Node looked: in the project folder, not in `src/`.

Common mistake. Leaving out `'utf8'`. Without an encoding, `readFileSync` returns a **Buffer** (the raw bytes), and `console.log` prints `<Buffer 5b 0a 20 20 7b ...>` instead of the text. Pass `'utf8'` whenever you want the file as a string.

npm and package.json

package.json: the project's manifest

`package.json` **describes a Node project: its name and version, the packages it depends on, and the scripts that start, develop and test it.**

The fields you will meet

Field	What it says	Example
name, version	The project's identity	"tasks-api", "1.0.0"
private	true: npm refuses to publish it by accident	true
type	How <code>.js</code> files are read (see ES modules)	"commonjs"
main	The entry file	"src/server.js"
scripts	Named commands for npm start, npm test, npm run <name>	"dev": "node --watch src/server.js"
dependencies	Packages the app needs to run	"express": "^4.22.3"
devDependencies	Packages needed only to develop and test	"jest": "^29.7.0"

Scripts

- **Named shortcuts** for terminal commands, listed under `"scripts"`.
- `npm start` **and** `npm test` run the start and test scripts; any other name needs run:
`npm run dev`.
- **Installed tools by name:** while a script runs, npm adds `node_modules/.bin` to the search path, so `"test": "jest"` works without a path.

JSON

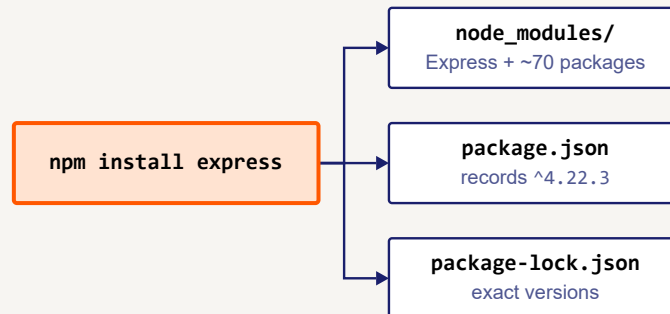
```
{
  "name": "tasks-api",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "node --watch src/server.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.22.3"
  },
  "devDependencies": {
    "jest": "^29.7.0",
    "supertest": "^7.0.0"
  }
}
```

Example. The usual split: start runs plain `node src/server.js`, what a production server runs, and dev adds `--watch` for development (`npm run dev`). A teammate finds in one place how to start your project.

Common mistake. Typing `npm dev` instead of `npm run dev`. Only `start`, `test`, `stop` and `restart` work without `run`; for any other name `npm` answers `Unknown command: "dev"`.

Installing packages: dependencies, devDependencies and node_modules

`npm install express` **downloads a package into `node_modules/` and records it in `package.json`, so anyone can install the same packages again.**



One command, three results: the code, the record and the exact versions.

Two lists in package.json

dependencies What the app needs to **run**: `npm install express`. A production server installs only this list (`npm ci --omit=dev`), so everything the running app requires must be here.

devDependencies What you need only to **develop and test**: `npm install --save-dev jest` (short form: `npm i -D jest`).

The `npm` commands you will use most

Command	What it does
<code>npm install</code>	Installs everything <code>package.json</code> lists (both lists)
<code>npm install express@4</code>	Adds Express 4 to dependencies; @4 picks the major version
<code>npm install --save-dev jest</code>	Adds jest to devDependencies
<code>npm uninstall express</code>	Removes it from <code>node_modules/</code> and from <code>package.json</code>
<code>npm ci</code>	Installs exactly what the lockfile lists (see package-lock.json)
<code>npm ls</code>	Shows what is installed

Example. Starting an API on Express 4: `npm init -y`, then `npm install express@4`. The @4 matters: a plain `npm install express` today installs **Express 5**, the current major version.

Common mistake. Committing `node_modules/`: thousands of files of other people's code, some built for **your** operating system and broken on a teammate's. Put `node_modules/` in `.gitignore` **before** the first commit. If it is already committed, `git rm -r --cached node_modules` removes it from the repository and keeps it on your disk.

Version numbers and ranges: what ^4.19.0 means

Packages use **semantic versioning**, `MAJOR.MINOR.PATCH`, and `package.json` stores a **range of versions you accept**, such as `^4.19.0`.

MAJOR: 5.0.0 Something changed in a way that **can break** your code.

MINOR: 4.20.0 New features, nothing removed: existing code keeps working.

PATCH: 4.19.1 Bug fixes only.

What each range lets `npm` install

Range	Accepts	Rejects
<code>^4.19.0 (caret)</code>	4.19.0 up to the newest 4.x.y	4.18.2, 5.0.0
<code>~4.19.0 (tilde)</code>	4.19.0 up to the newest 4.19.x	4.20.0

Range	Accepts	Rejects
4.19.0	Only 4.19.0	Everything else
^0.4.2	0.4.2 up to the newest 0.4.x	0.5.0
*	Any version at all	Nothing: avoid it

- **The caret is the default:** npm install writes ^, which means "compatible with": the same major version, at least this one.
- **Below 1.0 is different:** ^0.4.2 accepts only 0.4.x, because before 1.0 any minor release may break things.

```
// A simplified version of npm's caret rule, for versions >= 1.0.0
function caretAccepts(range, version) {
  const r = range.replace('^', '').split('.').map(Number);
  const v = version.split('.').map(Number);
  // another MAJOR may break your code
  if (v[0] !== r[0]) return false;
  if (v[1] !== r[1]) return v[1] > r[1]; // a newer MINOR is fine
  // same MINOR: PATCH must not be older
  return v[2] >= r[2];
}

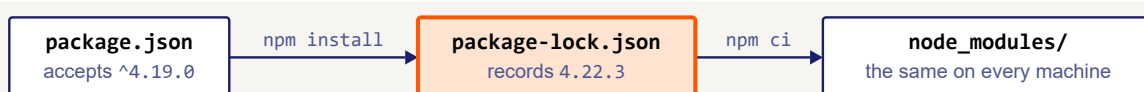
console.log(caretAccepts('^4.19.0', '4.22.3')); // true
console.log(caretAccepts('^4.19.0', '4.18.9')); // false: older
console.log(caretAccepts('^4.19.0', '5.0.0'));
// → false: new major
```

Example. With "express": "^4.19.2", a developer who installed last year and a teammate who installs today can get different 4.x versions, both allowed by the caret: compatible, but not identical. The **lockfile** records the exact version, so everyone gets the same one (see [package-lock.json](#)).

Common mistake. Reading ^4.19.0 as "version 4.19.0": it means "4.19.0 or any newer 4.x". The opposite slip is changing the major by hand ("^5.0.0") "to update": a new major may change behaviour your code relies on, so read the package's migration guide first.

package-lock.json, npm ci and what to commit

package-lock.json **records the exact version of every installed package, transitive ones included, so every machine can install the same set.**



The lockfile carries the exact versions from your machine to every other.

Two ways to install

	npm install	npm ci (clean install)
Use it	While developing	On servers, in CI, or to copy a teammate's exact setup
The lockfile	May update it (a new package, a changed range)	Never changes it
node_modules/	Updates it	Deletes it and installs exactly what the lockfile lists
Lockfile and package.json disagree	Updates the lockfile	Stops with an error

What goes into the repository

File or folder	Commit it?	Why
package.json	Yes	You write it: dependencies and scripts
package-lock.json	Yes	It pins the exact versions everyone must get

File or folder	Commit it?	Why
node_modules/	No: list it in .gitignore	Rebuilt by <code>npm ci</code> ; huge and partly specific to each operating system
.env	No: list it in .gitignore	Holds secrets (see .env files)
.env.example	Yes	Lists the variables needed, with placeholder values

```
# .gitignore for a Node project
node_modules/
.env
npm-debug.log*
.DS_Store
```

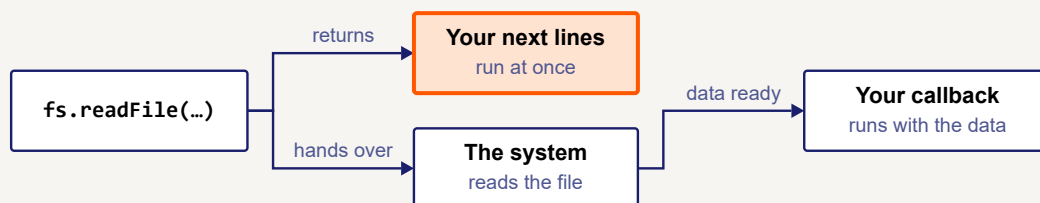
Example. A teammate clones the repo, runs `npm install`, and their server behaves differently from yours. The likely cause: the lockfile was never committed, so their install resolved every `^` range again and picked newer versions. With the lockfile in the repository, `npm ci` gives them the very same versions you have.

Common mistake. Deleting `package-lock.json` "to fix" an installation problem and committing the new one: that silently upgrades every transitive dependency at once. Delete `node_modules/` instead (it is disposable) and run `npm ci`; regenerate the lockfile only on purpose.

Asynchronous code

Blocking and non-blocking: why Node code is asynchronous

A **synchronous** (blocking) call waits until its work is done; an **asynchronous** (non-blocking) call starts the work, returns at once, and hands the result to a function **later**.



The call returns at once; the result comes back later, through the callback.

Two ways to call slow work

	Synchronous (blocking)	Asynchronous (non-blocking)
The call	Waits until the work is done	Returns at once; the next line runs
The result	The return value	Passed later to a callback, or through a promise
Examples	<code>fs.readFileSync</code> (names end in Sync)	<code>fs.readFile</code> , <code>setTimeout</code> , <code>fetch</code> , database queries
Fine for	One-off scripts; start-up, before the server listens	Everything a server does while requests arrive

```
const fs = require('node:fs');

// Blocking: the next line waits until the whole file has been read
const text = fs.readFileSync('notes.txt', 'utf8');
console.log('1. read', text.length, 'characters');

// Non-blocking: start reading, hand over a callback, carry on
fs.readFile('notes.txt', 'utf8', (err, data) => {
  console.log('3. the callback runs when the file has arrived');
});
console.log('2. this line runs before the file arrives');
```

Example. Two users call your API at the same moment. Request A runs a database query that takes 80 ms; request B just answers `{ status: 'ok' }`. With an asynchronous query, Node starts A's query, answers B at once and finishes A when the database replies. With a blocking query, B waits 80 ms behind A, and with 100 users the last one waits 8 seconds.

Common mistake. Expecting an asynchronous result on the next line: `let data; fs.readFile('notes.txt', 'utf8', (err, d) => { data = d; }); console.log(data);` prints undefined, because the `console.log` runs **before** the callback. Use the value inside the callback or, better, with `await` (see [async and await](#)).

Callbacks and the error-first convention

In Node's original style you pass a **callback** as the last argument, and Node calls it with `(err, result)` when the work is done.

err, first null on success; an Error object on failure. So every callback starts with `if (err) return ...`

result, second The value, such as the text of the file. undefined when there was an error.

```
const fs = require('node:fs');

// Three steps that depend on each other: "callback hell"
fs.readFile('config.json', 'utf8', (err, text) => {
  if (err) return console.error(err);
  const config = JSON.parse(text);
  fs.readFile(config.dataFile, 'utf8', (err, data) => {
    if (err) return console.error(err);
    fs.writeFile('copy.txt', data, (err) => {
      if (err) return console.error(err);
      console.log('done');
    });
  });
});
```

```
// A function written in Node's callback style (this one answers at once)
function divide(a, b, callback) {
  if (b === 0) {
    callback(new Error('Cannot divide by zero')); // failure: the error first
    return;
  }
  // success: null, then the result
  callback(null, a / b);
}

divide(10, 2, (err, result) => {
  if (err) return console.log('Error:', err.message);
  // Result: 5
  console.log('Result:', result);
});

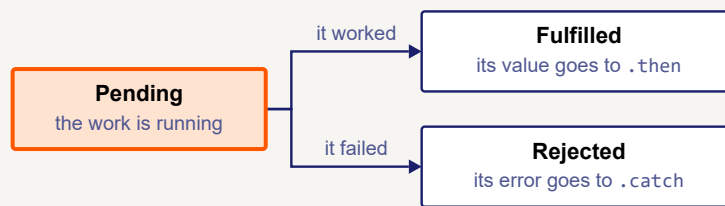
divide(1, 0, (err, result) => {
  if (err) return console.log('Error:', err.message); // Error: Cannot divide by zero
  console.log('Result:', result);
});
```

Example. `fs.readFile('missing.txt', 'utf8', (err, data) => { ... })`: the file does not exist, so `err` is an Error whose code is `'ENOENT'` ("no such file or directory") and `data` is undefined. `if (err) return console.error(err);` reports it and stops; without it, the next line would use the missing data and crash.

Common mistake. Leaving out the `return` in `if (err) callback(err);`: the function reports the error and then **carries on** with the success path, so the callback runs twice. Write `if (err) return callback(err);`. Also, a `try/catch` around `fs.readFile(..., cb)` does not catch the file error: it arrives later, after the `try` has finished (see [Errors in async code](#)).

Promises: a value that arrives later

A **promise** is an object that stands for a result that is not ready yet; it settles once, with a value or with an error.



A promise settles once, and never changes again.

- .then(fn)** Runs *fn* with the value. Returns a **new** promise, so calls chain instead of nesting.
- .catch(fn)** Runs *fn* with the error of any earlier step.
- .finally(fn)** Runs *fn* either way: clean-up.

How a chain passes values

- A **returned value** is passed to the next `.then`.
- A **returned promise** is waited for; its value is passed on.
- **One** `.catch` **at the end** handles a failure in any step.

```

const fs = require('node:fs/promises');

// The three dependent steps of the callback card, flat
fs.readFile('config.json', 'utf8')
  // a value: passed on
  .then((text) => JSON.parse(text))
  .then((config) => fs.readFile(config.dataFile, 'utf8')) // a promise: waited for
  .then((data) => fs.writeFile('copy.txt', data))
  .then(() => console.log('done'))
  .catch((err) => console.error('Something failed:', err.message)); // any step

// Making your own promise: wait ms milliseconds
function sleep(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}
sleep(500).then(() => console.log('half a second later'));
  
```

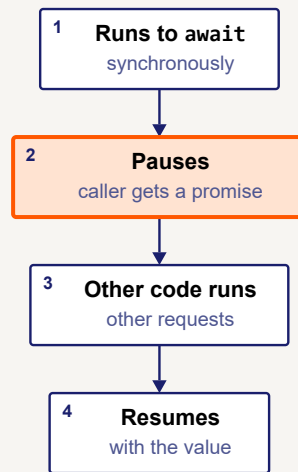
Example. `fetch('http://localhost:3000/health')` returns a pending promise at once. `.then((res) => res.json())` runs when the response arrives and returns another promise (reading the body); the next `.then((body) => console.log(body.status))` gets the parsed object. If the server is not running, the first promise is **rejected**: no `.then` runs, and the final `.catch` prints the error.

Common mistake. Starting more async work inside a `.then` without returning it: `.then((config) => { fs.readFile(config.dataFile, 'utf8'); })`. With braces and no `return`, the callback returns `undefined`, so the next `.then` runs at once instead of waiting, and a failed read escapes the final `.catch`. Write `return fs.readFile(...)`, or use the short arrow form without braces.

async and await

Inside an `async` function, `await promise` **pauses that function** until the promise settles, then gives back its

value; the code reads top to bottom, yet the thread is never blocked.



Only the async function waits; the thread keeps working.

- **Always a promise:** an async function returns a promise, even when it says return 5. Its caller must await it too, or use .then, which is why async spreads upwards to the callers.
- **Only inside async:** await works inside an async function, or at the top level of an ES module.
- **Same machinery as .then:** the rest of the function is queued as a **microtask** when the promise settles (see [Microtasks before tasks](#)).

```

const fs = require('node:fs/promises');

async function copyData() {
  const text = await fs.readFile('config.json', 'utf8'); // pauses here; the thread stays free
  const config = JSON.parse(text);
  const data = await fs.readFile(config.dataFile, 'utf8');
  await fs.writeFile('copy.txt', data);
  // fulfils the returned promise
  return data.length;
}

copyData().then((n) => console.log(`copied ${n} characters`));
console.log('this prints first: copyData() returned a pending promise');

```

```

// The same shape as copyData, with a fake slow read instead of the file system
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
async function readSlowly(name) {
  // pretend this is a disk read
  await sleep(200);
  return `contents of ${name}`;
}

async function copyData() {
  console.log('copyData starts');
  // pauses only copyData
  const text = await readSlowly('config.json');
  console.log('got:', text);
  return text.length;
}

copyData().then((n) => console.log('copied', n, 'characters'));
console.log('this prints second: copyData() returned a pending promise');

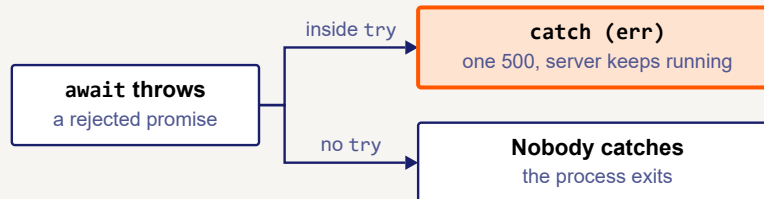
```

Example. The fetch of the promises card, with await: `async function checkHealth() { const res = await fetch('http://localhost:3000/health'); const body = await res.json(); console.log(body.status); }`. One await per promise, in the order the values are needed.

Common mistake. Forgetting `await`: `const task = db.findTask(id); console.log(task.title);` prints `undefined`, because `task` is a **promise**, which has no `title`. Sent as JSON, a promise becomes `{}`. When a value prints as `Promise { <pending> }` or arrives as `{}`, look for the missing `await`.

Errors in async code: try/catch every await

When an awaited promise is rejected, `await` **throws** the error on that line, so an ordinary `try { ... } catch (err) { ... }` catches it; an error nobody catches stops the whole server.



The same failure: one user gets an error, or every user loses the server.

```

const http = require('node:http');
// db stands for a database client whose queries return promises

const server = http.createServer(async (req, res) => {
  try {
    // rejects if the database is down
    const tasks = await db.tasks.findAll();
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(tasks));
  } catch (err) {
    // the details stay in the server log
    console.error(err);
    res.writeHead(500, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Something went wrong' }));
  }
});
  
```

Example. The database is restarting when a user asks for the tasks. Without the `try`, the terminal prints `Error: connect ECONNREFUSED` and the process **exits**: every later request fails until someone restarts it (`--watch` restarts on a file change, not after a crash). With it, this one user gets a 500 and the server keeps serving everyone else.

Common mistake. `try { saveTask(task); } catch (err) { ... }` **without** `await`. The call returns a promise at once, the `try` ends, and the rejection arrives after the net is gone. The `await` must be **inside** the `try`.

Independent work in parallel: Promise.all

`await Promise.all([a(), b()])` runs independent operations at the same time and waits once, so it takes about as long as the slowest one instead of their sum.

Two queries of 100 ms and 120 ms

	One after the other	In parallel
Code	<code>await a(); await b();</code>	<code>await Promise.all([a(), b()])</code>
Takes	<code>100 + 120 = 220 ms</code>	The slowest: 120 ms
Use it when	<code>b</code> needs the result of <code>a</code>	The operations are independent

- **The result** is one array of the values, in the order of the input array, not in the order they finished. `const [user, tasks] = await Promise.all(...)` takes them out by position (**array destructuring**).
- **One failure rejects it all**, at once, with that error. The other operations keep running, but their results are ignored. That suits a request that needs every piece.
- **Every outcome, even failures:** `Promise.allSettled` gives `{ status, value }` or `{ status, reason }` for each one.

```
async function loadDashboard(id) {
  // Sequential: about 100 ms + 120 ms = 220 ms
  // const user = await db.users.findById(id);
  // const tasks = await db.tasks.findByOwner(id);

  // Parallel: both start now; about max(100, 120) = 120 ms
  const [user, tasks] = await Promise.all([
    db.users.findById(id),
    db.tasks.findByOwner(id),
  ]);
  return { user, tasks };
}
```

```
// Fake queries: each answers after ms milliseconds
const sleep = (ms, value) => new Promise((resolve) => setTimeout(() => resolve(value), ms));
const findUser = () => sleep(100, { name: 'Ana' });
const findTasks = () => sleep(120, ['Buy milk', 'Study']);

async function sequential() {
  const t0 = Date.now();
  const user = await findUser();
  const tasks = await findTasks();
  console.log('sequential:', Date.now() - t0, 'ms', user.name, tasks.length);
}

async function parallel() {
  const t0 = Date.now();
  const [user, tasks] = await Promise.all([findUser(), findTasks()]);
  console.log('parallel:', Date.now() - t0, 'ms', user.name, tasks.length);
}

sequential().then(parallel);
```

Example. A dashboard needs the user, their tasks and their unread notifications: three queries of about 50 ms that each need only the user id. Awaiting them one by one answers in about 150 ms; `const [user, tasks, notes] = await Promise.all([...])` answers in about 50 ms, still inside one `try/catch`.

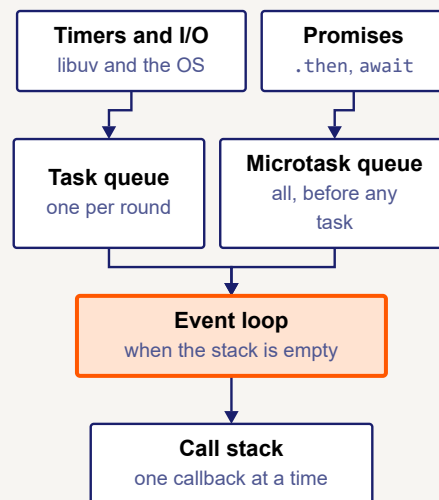
Common mistake. Writing `await` inside the array: `Promise.all([await a(), await b()])`. The `awaits` run one after the other **while the array is being built**, so the work is sequential again. Pass the promises themselves: `Promise.all([a(), b()])`.

The event loop

The event loop: one thread, a call stack and queues

The **event loop** is how one JavaScript thread handles many things: finished work queues a callback, and the

loop runs the callbacks one at a time, whenever the call stack is empty.



Callbacks wait in queues; the loop moves one to the stack only when it is empty.

One round of the loop

1. Run the script until the call stack is empty.
2. Run every microtask (promise callbacks, the code after an `await`), until the microtask queue is empty, including microtasks added meanwhile.
3. Take **one** task (an expired timer, a finished I/O operation) and run it to the end.
4. Back to step 2. When nothing is left to run or wait for, a Node program exits.

```

console.log('script start');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'));
console.log('script end');
// Predict the four lines, then press Run.
  
```

Example. In the Try it box, both plain `console.log` lines run while the script is on the stack. `setTimeout` hands its callback to the timer, which queues it as a task after 0 ms. `.then` on a fulfilled promise queues a microtask. When the script ends, the loop empties the microtask queue (promise), then takes the task (timeout). Step through it below with the "Promise.then versus setTimeout" program.

Common mistake. Reading `setTimeout(fn, 1000)` as "run fn in exactly one second". It means "queue fn **no sooner than** one second from now"; if the stack is busy or other tasks are ahead, it runs later. A timer delay is a minimum, not an appointment.

Microtasks before tasks: predicting the order

Asynchronous code prints in this order: all synchronous code; then every microtask, in the order queued; then tasks such as timers, one at a time, each followed by the microtasks it created.

Where each kind of callback waits

You write	It waits in	It runs
<code>console.log(...)</code> , any plain call	Nothing: it runs now	Immediately, on the call stack
<code>.then(fn)</code> , <code>.catch(fn)</code> , code after <code>await</code> , <code>queueMicrotask(fn)</code>	Microtask queue	As soon as the stack is empty, before any task
<code>setTimeout(fn, ms)</code> , <code>setInterval(fn, ms)</code>	Task queue, once the delay has passed	One at a time, after all microtasks
I/O callbacks: <code>fs.readFile</code> , an incoming request	Task queue, once the I/O has finished	One at a time, after all microtasks

A callback is queued only when it is ready

- **Chains:** in `Promise.resolve().then(a).then(b)`, `a` is queued at once, but `b` waits for the promise that `.then(a)` returned, so it is queued only after `a` has run.
- **async functions** run synchronously up to their first `await`; the rest is queued when the awaited promise is fulfilled.
- **Timers and I/O** are queued when the delay has passed or the I/O has finished, not when you write the line.

Two more queues, in Node only

`process.nextTick(fn)` Runs **before** the promise microtasks (in a CommonJS file).

`setImmediate(fn)` Runs in the loop's **check phase**, right after the phase that handles finished I/O. Inside an I/O or timer callback it runs before a new `setTimeout(fn, 0)`; at the top level of a script their order is not guaranteed.

```
console.log('1');
setTimeout(() => {
  console.log('2');
  Promise.resolve().then(() => console.log('3'));
}, 0);
Promise.resolve().then(() => {
  console.log('4');
  setTimeout(() => console.log('5'), 0);
});
console.log('6');
// Predict the order, then press Run. Then swap two lines and predict again.
```

Example. In the Try it box: 1 and 6 are synchronous. The `.then` callback is a microtask, so 4 comes next, and it schedules a second timer. The first timer's task prints 2 and queues a microtask, which runs before the next task: 3. Last, the second timer: 5. Result: 1, 6, 4, 2, 3, 5.

Common mistake. Ordering callbacks by the line they are written on. `setTimeout` on line 2 and `.then` on line 6 say nothing about which runs first: the queue a callback joins, and when it joins, decide.

Never block the event loop

Blocking the event loop means keeping the call stack busy with long synchronous work: no other callback can run, so in a server **every** other request waits.

What blocks

- **Long loops** over big data.
- `JSON.parse` **or** `JSON.stringify` of very large values.
- **Synchronous APIs in a request handler:** `fs.readFileSync`, `crypto.pbkdf2Sync`, `bcrypt.hashSync` for passwords.
- **Badly written regular expressions** on long input.

```
const http = require('node:http');

// One slow route freezes the whole server
http.createServer((req, res) => {
  if (req.url === '/slow') {
    const end = Date.now() + 5000;
    while (Date.now() < end) {}
    // → 5 s of busy waiting on the call stack
  }
  res.end(`done: ${req.url}`);
}).listen(3000);
// While /slow runs, a request to /health waits too: it is stuck in the queue.
```

Example. Start the server, run `curl http://localhost:3000/slow` in one terminal and straight away `curl http://localhost:3000/health` in another. `/health` normally answers in milliseconds; now it waits until the five seconds are over, because the event loop cannot take its request while `/slow` holds the stack. The visualiser's "A blocking loop delays a timer" program shows the same with a timer.

Common mistake. Thinking `async` makes a function non-blocking. `async function work() { for (let i = 0; i < 1e9; i++) {} }` still blocks for the whole loop: `async` only changes what the function **returns** (a promise). Code stops blocking only where it really waits for something outside the thread, at an `await`.

Configuration and app structure

Environment variables and `process.env`

An **environment variable** is a named setting handed to a program by whatever starts it, outside the code; Node exposes them in `process.env`.

- **Always text:** convert numbers with `Number(...)`, and compare flags as strings:
`process.env.DEBUG === 'true'`.
- undefined **when never set**, so give a default: `const PORT = process.env.PORT || 3000`; . Hosting platforms set `PORT` for you; on your laptop the default applies.

Setting a variable for one run

Terminal	Command
Git Bash, macOS, Linux	<code>PORT=4000 npm start</code>
PowerShell	<code>\$env:PORT = 4000; npm start</code> (stays set until you close the window)

```
// In Node this object would be process.env; here we fake it.
const env = { PORT: '4000', DEBUG: 'false' };

const port = env.PORT || 3000;
console.log(port, typeof port);
// → 4000 string

const portNumber = Number(env.PORT) || 3000;
console.log(portNumber, typeof portNumber); // 4000 number

if (env.DEBUG) console.log('debug is on?!'); // runs: 'false' is a non-empty string
console.log(env.DEBUG === 'true');
// → false: compare the text

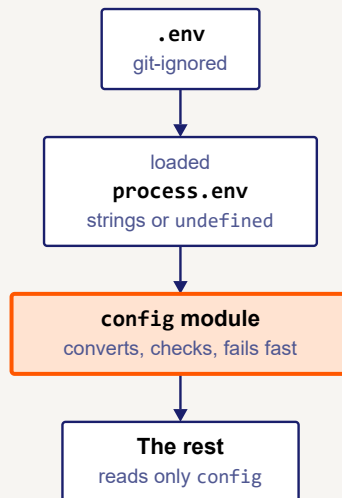
console.log(env.DATABASE_URL);
// → undefined: never set
```

Example. Port 3000 is taken on a teammate's laptop. Instead of editing `server.js` (and fighting over it in every merge), they run `PORT=4000 npm start`. `process.env.PORT` is `'4000'`, the server listens on 4000, and the code in Git never changed.

Common mistake. Testing a flag with `if (process.env.DEBUG)` when it is set to `false`. The value is the **string** `'false'`, which is **truthy** (every non-empty string is), so debug mode switches on. The same trap with numbers: `process.env.PORT + 1` is `'40001'`, not `4001`.

`.env` files, config and keeping secrets out of Git

A `.env` file lists a project's environment variables, one `NAME=value` per line; it holds secrets, so it never goes into

Git.

Secrets stay out of Git and enter the app through one door.

.env The real values. Listed in .gitignore, never committed.

.env.example The same names with placeholder values. Committed, so a teammate knows what to fill in.

Loading it, before anything reads it

- `node --env-file=.env src/server.js`: built into Node 20.6 and later, nothing to install.
- `require('dotenv').config();` on the first line of the entry file, after `npm install dotenv`.
- **The real environment wins**: a variable already set, for example by a hosting platform, is not replaced by the file.

```

// .env.example (committed)
// .env (git-ignored)
// PORT=3000
// PORT=3000
// DATABASE_URL=postgres://ana:s3cret@localhost:5432/tasks
// DATABASE_URL=postgres://user:password@localhost:5432/tasks
//
// JWT_SECRET=change-me
// JWT_SECRET=k8Jq2v...

// src/config/index.js: the only file that reads process.env
function required(name) {
  const value = process.env[name];
  if (!value) throw new Error(`Missing environment variable ${name} (see .env.example)`);
  return value;
}

module.exports = {
  port: Number(process.env.PORT) || 3000,
  databaseUrl: required('DATABASE_URL'),
  jwtSecret: required('JWT_SECRET'),
};

// src/server.js: load .env FIRST, before anything reads the config
// or start with: node --env-file=.env src/server.js
require('dotenv').config();
const config = require('./config');
const app = require('./app');
app.listen(config.port, () => console.log(`Listening on http://localhost:${config.port}`));

```

Example. A new teammate clones the repo, runs `npm ci` and `npm start`, and gets `Error: Missing environment variable DATABASE_URL` (see `.env.example`) at once. They copy the example (`cp .env.example .env` in Git Bash, `Copy-Item .env.example .env` in PowerShell), fill in their own password and start again. Without fail-fast, the server would start "fine" and crash on the first request that touched the database.

Common mistake. Loading `.env` too late. If `server.js` requires `./config` **before** `require('dotenv').config()`, the config module runs first, finds nothing, and reports a missing variable that is right there in `.env`. Load it on the first line, or use `--env-file`. And never "fix" a missing variable by pasting the real secret into the code as a default.

Why a framework, and why Express

A **web framework** does the chores every server repeats, so each route is one line instead of one more `if`; the examples on this site use **Express**.

Routing Method + path → a function: `app.get('/tasks', listTasks)`.

Middleware Functions every request passes through, in order: a logger, a body parser, a login check.

Structure Conventions most projects share (see [A project layout](#)).

Four Node frameworks you will hear about

Framework	Style	Typical use
Express	Minimal, unopinionated, a huge choice of middleware	Small and medium APIs; the worked example on this site
Fastify	Very fast; validates requests against schemas; plugins	APIs where performance matters
Koa	Minimal; middleware written with <code>async/await</code>	Lightweight custom servers
NestJS	Structured, TypeScript first, many built-in conventions	Large applications built by big teams

```
// The server of "A first server", with Express
const express = require('express');
const app = express();

// one line per route
app.get('/health', (req, res) => res.json({ status: 'ok' }));
// Any other path: Express answers 404 by itself

app.listen(3000, () => console.log('Listening on http://localhost:3000'));
```

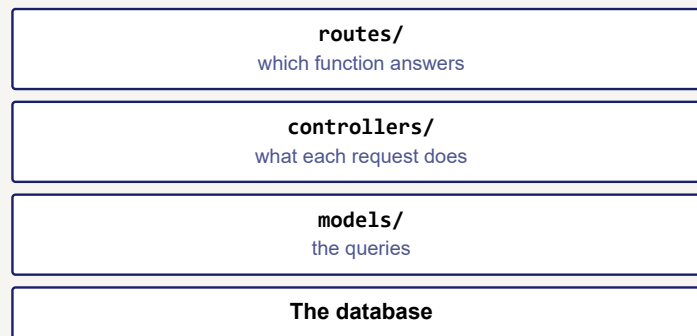
Example. Adding `GET /tasks/:id` to the plain http server means splitting `req.url` by hand, checking the method, converting the id and repeating the `Content-Type` header. In Express it is `app.get('/tasks/:id', getTask)`, and `req.params.id` already holds the id. Multiply that by twenty routes.

Common mistake. "Express replaces Node." Express is a package that **runs inside** Node: installed with `npm`, loaded with `require('express')`, built on the `http` module. Everything in this section (modules, `npm`, the event loop, `process.env`) applies to every Express app.

A Node/Express project layout

A typical Node API keeps its code in `src/`, one folder per job, and splits building the app (`app.js`) from starting it

(server.js).



Each folder calls only the one below it.

routes/	Which function answers each method and URL.
controllers/	What each request does: read req, do the work, send res.
models/	The database queries, once there is a database.
middleware/	Steps shared by many requests: logging, checking a login, the error handler.
config/	Reads process.env in one place (see .env files).
app.js, server.js	app.js builds and exports the app; server.js requires it and calls listen. Tests can then load the app without opening a port (see app.js builds , server.js starts).

Folder tree

```

my-api/
├─ package.json
├─ package-lock.json
├─ .gitignore      # node_modules/ and .env
├─ .env            # real settings: NOT committed
├─ .env.example    # same keys, placeholder values: committed
├─ README.md
├─ src/
│  ├─ server.js    # requires app, calls app.listen()
│  ├─ app.js       # builds and exports the Express app
│  ├─ routes/      # method + URL → controller function
│  ├─ controllers/ # request logic: read req, send res
│  ├─ models/      # database queries (once there is a database)
│  ├─ middleware/  # logger, auth, error handler
│  └─ config/      # reads process.env in one place
└─ tests/         # automated tests
  
```

Example. Follow GET /tasks: server.js started the app; app.js ran the shared middleware (the logger, the body parser) and passed the request to routes/tasks.js; the route called listTasks in controllers/tasksController.js, which asked models/task.js for the rows and sent them as JSON. Five files, one small job each.

Common mistake. Calling app.listen() inside app.js "because it is shorter". It works until the first test file requires the app: every test run then opens a real port, and a second test file fails with EADDRINUSE. Keep the rule strict: app.js exports, server.js listens.